

Reliability & Resilience

Discuss the patterns used to handle failures in microservices.

A: Common patterns include **Timeouts** (to prevent waiting indefinitely), **Retries** (to recover from temporary faults), **Circuit Breaker** (to stop calling failed services), and **Bulkhead** (to isolate resources). These patterns increase the application's tolerance to external failures and prevent cascading errors.

What is the Circuit Breaker pattern?

A: The **Circuit Breaker pattern** is a mechanism used to prevent cascading failures. It monitors calls to an external service; if the failure rate exceeds a threshold, it "opens the circuit" and immediately fails subsequent calls, returning a quick error response. This gives the failing service time to recover and prevents the upstream service from exhausting its resources.

How does the Bulkhead pattern help in improving system resilience?

A: The **Bulkhead pattern** helps by isolating the resources (like thread pools or connection pools) used to call different external services. If one service fails and exhausts its dedicated pool, other services remain unaffected. This prevents one failing component from bringing down the entire application by consuming all shared resources.

Can you explain the Retry and Backoff patterns?

A: The **Retry pattern** automatically attempts a failed operation again. The **Backoff pattern** is used with retry to increase the wait time between each attempt (e.g., 1s, then 2s, then 4s). This prevents the retries from immediately overwhelming an external service that is struggling to recover.

Scenario: A microservice might fail : how would you implement Circuit Breaker?

A: I would implement a Circuit Breaker using a library like **Resilience4J** and wrap the service call inside a dedicated method annotated with `@CircuitBreaker`. I would configure thresholds (e.g., 50% failure rate over 10 calls) and define a **fallback method** that executes immediately when the circuit opens, ensuring the user gets a graceful response instead of waiting.

Distributed Data & Transactions

How would you implement a transaction that spans multiple services?

A: I would implement a distributed transaction using the **Saga Pattern**. This involves breaking the global transaction into a sequence of small, local transactions in different services. Each local transaction publishes an event to trigger the next step.

How do you address the issue of data consistency across microservices?

A: Data consistency is addressed using **Eventual Consistency**. When one service changes its data, it publishes an event via a message broker. Other interested services consume that event and update their local data asynchronously, ensuring all service data eventually aligns.

Let's say you have to choose between DB per service or shared DB, which and why?

A: I would always choose **DB per service**. While a shared DB is easier initially, it creates **tight coupling** and prevents services from deploying and scaling independently. A DB per service enhances **autonomy** and allows each service to choose the best database technology for its specific needs.

How do you handle distributed transactions? Explain the Saga pattern.

A: The **Saga pattern** is a decentralized way to handle transactions across services. It is a sequence of local transactions, where every transaction is coupled with a **compensation action**. If any local transaction fails, the compensation actions are executed in reverse order to undo the previously completed transactions.

If a Saga step fails, how do you ensure consistency? Describe compensation actions.

A: If a step fails, consistency is ensured by triggering a **compensation action**. A compensation action is an additional transaction designed specifically to undo the changes made by a previous successful transaction (e.g., if a payment fails, the compensation action would be to release the reserved inventory).

Deployment & Scaling

You notice that one microservice is experiencing high load, how do you scale it?

A: I would scale the microservice **horizontally** by increasing the number of running instances (or Kubernetes pods) dedicated to that service. I would also examine the database and cache layers, as they are often the bottleneck, and tune their connection pools or capacity accordingly.

How will you scale single/multiple microservices, is application.yml enough?

A: **application.yml** is **not enough** for scaling. It is only for configuration. To scale, you must use an **orchestration platform** like **Kubernetes**. You define rules (like **Horizontal Pod Autoscaler**) in Kubernetes configuration that automatically increase or decrease the number of running instances based on real-time metrics like CPU usage or request queue length.

What deployment strategies would you use for microservices?

A: I would use **Canary Releasing** and **Blue-Green Deployment**. Blue-Green offers instant rollback by switching traffic entirely to a new environment. Canary releasing is safer for critical changes, rolling out the new version to a small percentage of users (e.g., 5%) for monitoring before full release.

How do you mitigate risk during deployment (blue-green, canary)?

A: Risk is mitigated by minimizing the blast radius of any bug. **Blue-Green** mitigates risk by keeping a stable, old environment running for instant rollback. **Canary** mitigates risk by limiting exposure of the new version to a small, non-critical user group while continuously monitoring performance and error rates.

Tracing & Observability

How can distributed tracing help in monitoring microservices?

A: Distributed tracing assigns a unique **Trace ID** to every request entering the system. As the request travels across multiple services, the Trace ID is preserved. This helps monitoring by showing the **full path** a request took, revealing where time was spent, and quickly pinpointing which service caused an error or performance bottleneck.

How can you track the flow of requests across multiple microservices?

A: You track the flow using **Correlation IDs**. Every service logs the unique **Trace ID** that was passed to it. By aggregating logs centrally and searching for that single Trace ID, you can view all related log entries from all services, stitching together the complete sequence of execution.

What monitoring & logging strategies would you employ?

A: I would employ a **Centralized Logging** strategy (ELK or Loki) to collect all service logs in one place. For monitoring, I would focus on **Golden Signals** (Latency, Traffic, Errors, Saturation) by collecting metrics using **Prometheus** and visualizing them with **Grafana**.

How would you centralize logs and metrics?

A: I would centralize **logs** by running a **log shipper** agent (like Filebeat or Fluentd) alongside each service to forward structured JSON logs to a cluster (like Elasticsearch). **Metrics** are centralized using **Micrometer** to record data, which Prometheus then scrapes and stores.

Security

How can services securely communicate with each other?

A: Services should communicate securely using **Mutual TLS (mTLS)**, where both the client and server services verify each other's identity using certificates. This ensures both **authentication** (verifying who they are) and **encryption** (securing the data in transit) for all internal traffic.

What are the implications of service-specific databases on security?

A: The main implication is **improved security isolation**. A compromise of one service does not immediately grant access to the entire system's data. Each service's secrets (database credentials) are restricted only to its local database, limiting the blast radius of any security breach.

What strategies secure service-to-service communication?

A: Strategies include implementing **mTLS** for strong identity verification. The services should also enforce **scoped authorization** by passing a **JWT** that defines exactly what the calling service is allowed to do (e.g., read-only access to a specific resource).

What are the security challenges in microservices architecture?

A: Challenges include managing **distributed secrets** (passwords, keys), securing a much larger **network attack surface**, and ensuring proper **authentication** between dozens of services. This is addressed by centralizing secrets (Vault) and traffic control (API Gateway).

Communication & Messaging

Can you explain the role of message brokers in microservices?

A: The **message broker** (like Kafka or RabbitMQ) provides an asynchronous communication backbone. Its role is to decouple services by acting as an intermediary: services publish events without knowing who the consumers are, and consumers read events without knowing who the producers are. This increases resilience.

What are the trade-offs of synchronous vs asynchronous communication?

A: **Synchronous** is simpler to implement and provides immediate feedback but risks **cascading failures** and tight coupling. **Asynchronous** promotes **high resilience and loose coupling** but adds complexity in debugging and requires robust mechanisms (like the Saga pattern) to handle distributed transactions.

How do different services communicate?

A: Services communicate using **HTTP/REST** (synchronous, request-response) for queries and data retrieval. They use **message brokers** (asynchronous, event-driven) for notifications, state changes, and long-running processes.

How would you handle the scenario where Kafka messages must be consumed by multiple services?

A: I would design each consuming service to belong to a **separate, independent consumer group**. This ensures that every service receives and processes its own copy of all messages published to that single Kafka topic, allowing independent processing for different business needs.

Spring Cloud Details

Explain a few microservices design patterns you're aware of.

A: I am familiar with the **API Gateway pattern** (single entry point), the **Service Discovery pattern** (finding services by name), and the **Circuit Breaker pattern** (handling external failures gracefully).

How do you implement service discovery in microservices?

A: I implement service discovery using a tool like **Eureka** or **Consul**. Service instances register their location with the registry. A client service then uses a **discovery client** to look up the instance by name before making an HTTP request.

Discuss differences between client-side and server-side discovery.

A: **Client-side discovery** (e.g., Eureka) requires the client service to query the registry and use a smart load balancer (like Spring Cloud's Load Balancer) to choose the correct instance. **Server-side discovery** (e.g., Kubernetes Ingress) moves the discovery and routing logic entirely to the load balancer/gateway. Client-side is often faster but more complex for the client.

What are the components of a typical Spring Cloud architecture?

A: A typical architecture includes a **Service Registry** (like Eureka), a **Configuration Server** (for centralized settings), a **Gateway** (for routing and authentication), and **Client Libraries** (like OpenFeign) for resilient, load-balanced communication.

Docker & K8s Basic

What is the difference between Docker and Kubernetes?

A: Docker is a tool for **containerization**, packaging an application into a portable unit. **Kubernetes (K8s)** is an **orchestration system** for managing, deploying, scaling, and networking multiple Docker containers automatically across a cluster of machines.

If you were tasked with deploying a microservices application, how would you decide between Docker, Kubernetes, or both?

A: I would use **both**. Docker is necessary for packaging each individual microservice application as a container. Kubernetes is then necessary for the operational side, handling the automatic deployment, scaling, health monitoring, and internal networking for the entire ecosystem of containers.

JAVA Interview Buddy